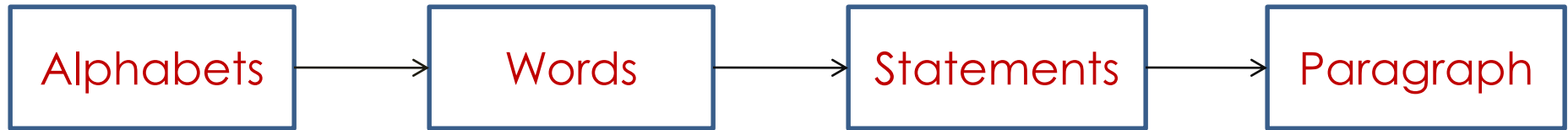


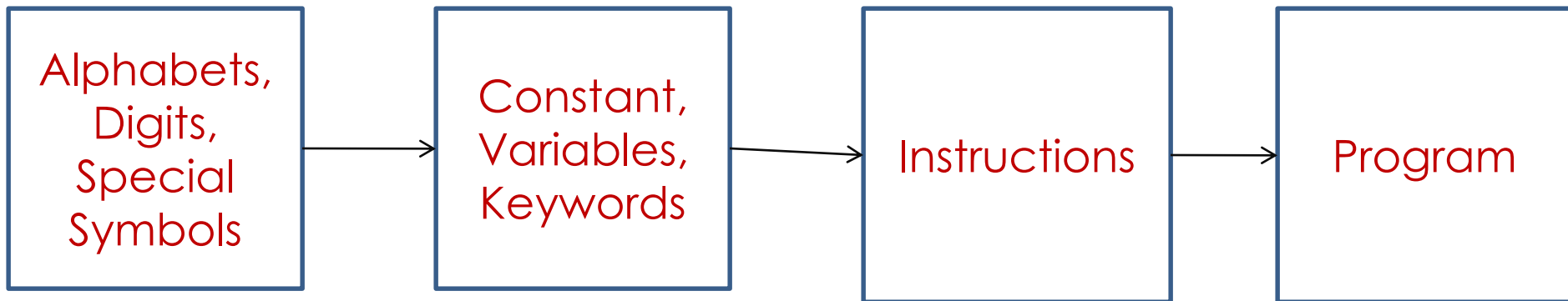
Chapter 2

Constants , Variables and Data Types

Steps for learning English Language



Steps for learning C Language



Character sets

Characters can be used to form words, numbers and expressions to instruct the computer.

ANSI C supports ASCII character set.

Hence generally recommended to use ASCII keyboard.

Characters are grouped into following categories.

- a) Letters : a to z ; A to Z
- b) Digits : 0 to 9
- c) Special Characters : # , ! , ~ , & , ...etc
- d) White spaces : Spacebar , tab
newline < enter> , ...etc.

Character sets

Table 2.1 C Character Set

Letters		Digits	
Uppercase A.....Z		All decimal digits 09	
Lowercase a.....z			
Special Characters			
,	comma	&	ampersand
.	period	^	caret
;	semicolon	*	asterisk
:	colon	-	minus sign
?	question mark	+	plus sign
'	apostrophe	<	opening angle bracket (or less than sign)
"	quotation mark	>	closing angle bracket (or greater than sign)
!	exclamation mark	(	left parenthesis
	vertical bar	)	right parenthesis
/	slash	[	left bracket
\	backslash	]	right bracket
~	tilde	{	left brace
_	under score	}	right brace
\$	dollar sign	#	number sign
%	percent sign		
White Spaces			
Blank space			
Horizontal tab			
Carriage return			
New line			
Form feed			

TRIGRAPH CHARACTERS

Some keyboards do not support special characters as ~ , # , [,] , \

They are represented by a special group of characters referred as TRIGRAPH characters.

e.g. ??= for #

 ??- for ~ ... and more

Table 2.2 *ANSI C Trigraph Sequences*

<i>Trigraph sequence</i>	<i>Translation</i>
??=	# number sign
??(	[left bracket
??)	] right bracket
??<	{ left brace
??>	} right brace
??!	vetical bar
??/	\ back slash
??/	^ caret
??-	~ tilde

C TOKENS

C has very rich character set, using which one can write very complex program in C. However the smallest individual elements/units in a program are called tokens.

The C tokens can be further divided into the following

- a) Keywords
- b) Identifiers
- c) Constants
- d) Strings
- e) Special Symbols
- f) Operators

C TOKENS ... examples

Keywords	<i>float , while</i>
Identifiers	<i>amount , name , roll_no</i>
Constants	<i>-15.5 , 100</i>
Strings	<i>“ABC” , “Hello” , “1000”</i>
Special Symbols	<i>[] , { }</i>
Operators	<i>+ , * , / , - , %</i>

A C program is written using these tokens, white space, and the syntax of language.

Each token should be separated from the other by a space, tab or carriage return.

```
printf ("HELLO");
```

is valid, even though white space has been omitted between the reserved word *printf* and the opening parentheses.

KEYWORDS

A word in C statement is classified as either a keyword or an identifier.

The keywords implement specific language feature. They are explicitly reserved words and cannot be used as names for the program variables or other user defined program elements.

eg. auto, int, case , if , for , const ... etc.

- * All keywords (32) are in lowercase
- * They are not identifiers.

IDENTIFIERS

Identifiers refers to the name of variables, functions, arrays, structure etc. given by the programmers, which is going to occupy a specific amount of memory.

Naming Convention for identifiers

1. Only Alphabetic characters, digits and underscores are permitted
2. The name can not start with a digit.
3. Uppercase and lowercase letters are distinct (Case sensitive).
4. A declared keyword cannot be used as an identifier name.
5. ANSI C standard places no limit on its length.
6. Although many implementation of C impose such restrictions. (8 – 31) e.g. `average_height` and `average_weight` both are same
7. Must not contain the white space

Exercise ...

Identify valid/invalid *identifiers* giving reasons

John

123

25th

seventh

float

Char

price\$

int_type

Solution ...

Identify valid/invalid *identifiers* giving reasons

John	valid
123	invalid
25th	invalid
Seventh	valid
float	invalid
Char	valid
price\$	invalid
int_type	valid

CONSTANTS

Constant in C refer to a fixed value, do not change during an execution of program. Based on the fixed value , which they hold they are classified into following categories.

a) Numeric Constant

1. Integer Constant : Integer value
2. Real Constant : Real value with fractional part.

b) Character Constant

1. Single character Constant
2. String Constant (Multiple Characters ..)

Example ... integer constant

Valid

123

-123

0

654321

Invalid

15 750

2,347

\$1000

4.0

For 32 bit machine

**EMBEDDED SPACES, COMMAS AND
NON DIGITS ARE NOT PERMITTED**

56789u (unsigned integer)

987645L (signed long integer)

Example ... integer constant

An Octal integer constant : (integer value in octal data format)

Represented with a leading 0 [numeric zero]

Examples

037

0435

An hexadecimal integer constant :

(integer value in hexadecimal data format)

Represented with a leading 0x / 0X [numeric zero and x/X]

0x34

0Xaf

Real constants

Constant value with some fractional part
Used when precision matters.

Examples

215.000083

-.89

+247.0

exponential notation / **floating point form**

mantissa **e** *exponent* or *mantissa* **E** *exponent*

-4.5e-2 representation of -0.045

+5.45678E3 representation of +5456.78

Exercise ...

Identify valid/invalid *numeric constant* giving reasons

698354L

25,000

7.1e 4

\$255

0X7B

1.5E+2.5

23.004

023.3400

Solution ...

Identify valid/invalid *numeric constant* giving reasons

698354L	Valid
25,000	Invalid
7.1e-4	Valid
\$255	Invalid
0X7B	Valid
1.5E+2.5	Invalid
23.004	Valid
023.3400	Valid

Character Constants

A character constant contains a single character enclosed within a pair of single quote marks.

e.g. 'X' , ';' , '5' ... etc

Note : A character constant '5' is not the same as number 5.

Each character constant has integer value , known as an ASCII value.

ASCII value of '5' is not equivalent to number 5.

How to get an ASCII value of a character ? ... will see later

please refer to an APPENDIX – II at the end of textbook.

String Constants

A string constant contains is a sequence of characters enclosed in double quotes.

e.g. “Hello” , “12345” , “X” , “5” ... etc

A string constant “X” is not the same as a character constant ‘X’.

Each character constant has integer value , known as an ASCII value, while the string constant does not have any ASCII value associated with it.

HW : Differentiate between 5 , ‘5’ and “5”, with reference to C.

Backslash Character Constants

C supports some special character constants
Represented along with backslash notation
Generally used in output functions.

<code>'\b'</code>	for back space
<code>'\n'</code>	for new line
<code>'\t'</code>	for horizontal tab
<code>'\''</code>	for single quotation mark
<code>'\?'</code>	for question mark
<code>'\\'</code>	for backslash

Backslash Character Constants

Table 2.5 Backslash Character Constants

<i>Constant</i>	<i>Meaning</i>
<code>'\a'</code>	audible alert (bell)
<code>'\b'</code>	back space
<code>'\f'</code>	form feed
<code>'\n'</code>	new line
<code>'\r'</code>	carriage return
<code>'\t'</code>	horizontal tab
<code>'\v'</code>	vertical tab
<code>'\''</code>	single quote
<code>'\"'</code>	double quote
<code>'\?'</code>	question mark
<code>'\\'</code>	backslash
<code>'\0'</code>	null

Write single *printf()* statement for displaying the following output on monitor.

- a) “Hello”
- b) How are you ?
- c) \\ invalid syntax for comment in C
- d) // single line comment in C

Predict the output for the following *printf()* statement in a C program.

```
printf(“Good morning sir\b\b\bstudent”);
```

Variables

- A variable is a data name given to data value.
- Its value can change any time during the execution.
- A variable name should be chosen by the programmer in meaningful way so that its use is reflected in a useful way in the entire program.

Examples

Sun

Valid

number

Valid

123

Invalid

Emp_name

Valid

area\$

Invalid

group one

Invalid

Salary

Valid

Average_number

Valid

%abc

Invalid

DATA TYPES IN C

Data refers to a piece of information.

Data may be stored in different ways.

They basically demand for

- a) storage space – memory generally in bytes
- b) Interpretation of the each bit

Define storage representations for information.

DATA TYPES IN C

1. Primary (or fundamental) Data Types

Basic data blocks to store information

integer (int /long) , real (float / double)

character (char)

2. User Defined Data Types

3. Derived Data Types

Derived from the Primary data types as a large block

Primary Data Types

a) To store integer value

Type	Memory Size	Range
int	2 bytes (16 bits)	- 32768 to 32767
unsigned int	2 bytes (16 bits)	0 to 65535
short int	1 byte (8 bits)	- 128 to 127
long int	4 bytes (32 bits)	$\pm 2^{31}$

b) To store real value

Type	Memory Size	Range
float	4 bytes (32 bits)	3.4E±38
double	8 bytes	1.7E±308
long double	10 bytes	3.4E±4932

Primary Data Types

c) To store character value

Type	Memory Size	Range
char	1 byte (8 bits)	-128 to 127
ASCII value of a Character		

Using these data types now we can request compiler to allocate memory for storing information.

Variable Declaration in C

handling of numeric values

a) int data type (value without fractional part)

use appropriate data type to accommodate the largest possible value.

b) float data type (value with fractional part)

use appropriate data type to accommodate the largest possible value with expected precision.

float - single precision

double - double precision

Example

a) *int* sum ;

A valid C statement which declare a variable,
with data type **integer** , identifier **sum**

In general *datatype* <identifier> ;

Datatype informs compiler about the **memory requirement** as well as the **data structure** in the requested bytes of memory block.

Identifier gives **name** to the allotted memory location, using which a programmer can access (read / write) the content (value) of that memory location.

Use of printf()

- It is used for displaying a value stored in a variable.
- The general format/Syntax of printf()

```
printf("format specifier", variable);
```


Format specifier

for various data types

%d for integer

%u for unsigned integer

%X for integer in hexadecimal notation

%ld for long integer

%f for float

%lf for double (long float)

%c for char

%s for string

Use of printf()

.

- - - -

- - - -

`int a ;` Declaration of an integer variable with name **a**

`a = 1000 ;` Assigns a value to **a**

`printf(“%d”,a);` Displays content of **a**
(using a format specifier of an integer)

- - - -

- - - -

Use of printf()

.

- - - -

- - - -

```
char b ;
```

```
b = 'x' ;
```

```
printf("%c",b);
```

- - - -

- - - -

- - - -

- - - -

```
float c ;
```

```
c = 23.45 ;
```

```
printf("%f",c);
```

- - - -

- - - -

```
#include <stdio.h>
```

```
int main()
```

```
{
```

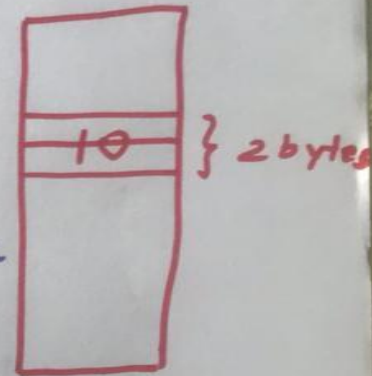
```
    int x; // variable declaration
```

```
    x = 10; // Assigning value to variable
```

```
    printf("%d", x);
```

```
    return 0;
```

```
}
```



```
int x;  
x = 10;
```

```
} int x = 10;  
    initialization
```

(At the time of declaration
assigning value to
variable)

Use of printf()

- It is also used to print the address of variables.

```
int x;
```

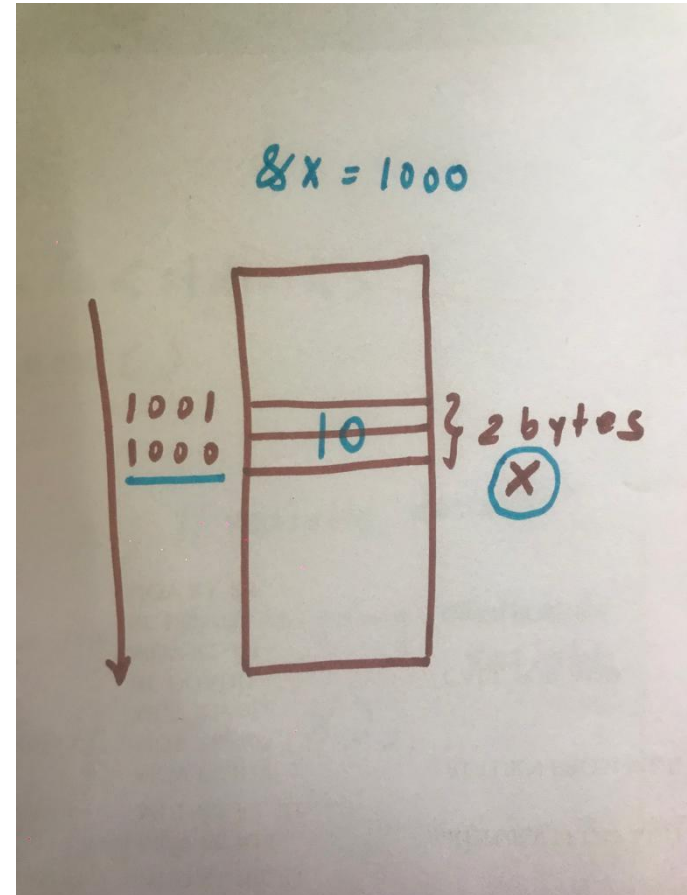
```
x=10;
```

```
printf("%u",&x);
```

Output:

1000

Note: Address of variable is
the address of lower byte



Simple Programs

- a) Write a C program to add values stored into two integer variables and display the result on screen.
- b) Write a C program to multiply values stored into two integer variables and display the result on screen.
- c) Write a program to display ASCII value of specific character on screen.

Solution (a)....

```
#include <stdio.h>
int main()
{
    int a,b,c; // int a=2,b=3,c;
    a=2;
    b=3;
    c=a+b;
    printf("The value of c=%d",c);
    //printf("The sum of %d and %d = %d",a,b,c);
    return 0;
}
```

Output:

The value of c=5

//The sum of 2 and 3 = 5

Reading Data from keyboard

- `scanf( )` is used to read the data from keyboard.
- The general format/Syntax of `scanf()`

`scanf("control string", &variable);`

Reading Data from keyboard

```
#include <stdio.h>
int main()
{
    int a;
    scanf("%d",&a);
    printf("%d",a);
    return 0;
}
```

Output:

2 (Enter)

2

Reading Data from keyboard

```
#include <stdio.h>
int main()
{
    int a,b;
    scanf("%d%d",&a,&b);
    printf("a=%d and b=%d",a,b);
    return 0;
}
```

Output:

2 3(Enter)
a=2 and b=3

2 (Enter)
3 (Enter)
a=2 and b=3

2 3 4(Enter)
a=2 and b=3

Reading Data from keyboard

```
#include <stdio.h>
int main()
{
    int a,b,c;
    scanf("%d%d",&a,&b);
    printf("a=%d and b=%d",a,b);
    scanf("%d",&c);
    printf(" c=%d",c);
    return 0;
}
```

Output:

2 3(Enter)
a=2 and b=3
4
c=4

2(Enter)
3(Enter)
a=2 and b=3
4
c=4

2 3 4(Enter)
a = 2 and b = 3
c=4

Reading Data from keyboard

```
#include <stdio.h>

int main()
{
    int a,b,c;
    printf("Enter the value of a and b\n");
    scanf("%d%d",&a,&b);
    printf("a=%d and b=%d\n",a,b);
    printf("Enter the value of c\n");
    scanf("%d",&c);
    printf(" c=%d",c);
    return 0;
}
```

Output:

1. Enter the value of a and b

2

3

a=2 b=3

Enter the value of c

4

c=4

2. Enter the value of a and b

2 3

a=2 b=3

Enter the value of c

4

c=4

3. Enter the value of a and b

2 3 4

a=2 b=3

Enter the value of c

c=4

Reading Data from keyboard

```
#include <stdio.h>
int main()
{
int x;
char y;
float z;
printf("Enter the value of integer x,
character y and float z\n");
scanf("%d%c%f",&x,&y,&z);
printf("x=%d y=%c z=%f",x,y,z);
return 0;
}
```

Output:

1.Enter the value of integer x, character y and float z

2a3.4

x=2 y=a z=3.400000

2.Enter the value of integer x, character y and float z

2

a

x=2 y=

z=0.000000

3.Enter the value of integer x, character y and float z

2 a 3.4

x=2 y= z=0.000000

Reading Data from keyboard

```
#include <stdio.h>
int main()
{
int x;
char y;
float z;
printf("Enter the value of integer x, character y and float z\n");
scanf("%d %c %f",&x,&y,&z);
printf("x=%d y=%c z=%f",x,y,z);
return 0;
}
```

- Write an appropriate scanf() for read the date given in following format.

27/10/2021

User Defined Type Declaration

- C supports a feature known as “type definition” that allows users to define an identifier that would represent an existing data type.
- The user defined data types identifier can later be used to declare variables.
- The general form:

`typedef type identifier;`

User Defined Type Declaration

- Examples:

```
typedef int x;
```

```
x a;
```

```
typedef float y;
```

```
y b;
```

User Defined Type Declaration

- Another user defined data type is enumerated data type provided by ANSI standard.
- It is defined as follows:

`enum identifier {value1,value2,.....,valuen};`

- Example:

`enum day {Monday,Tuesday,.....,Sunday};`

`enum day week_st,week_end;`

`Week_st =Monday;`

`Week_end=Sunday;`

Defining Symbolic Constant

- Symbolic Constant is a name that substitutes for a sequence of characters or a numeric constant, a character constant or a string constant.
- When program is compiled each occurrence of a symbolic constant is replaced by its corresponding character sequence.
- It provides
 - 1) Modifiability
 - 2) Understandability

Defining Symbolic Constant

- The general syntax of symbolic constant is:
`#define symbolic-name value of constant`

- Examples:

```
#define MAX 50
```

```
#define TRUE 1
```

```
#define FALSE 0
```

```
#define SIZE 15
```

```
#define PI 3.14
```

Defining Symbolic Constant

- Identify the invalid symbolic constants:

`#define X=2.5`

`# define TRUE 1`

`#define N 25;`

`#define M 5, N 10`

`#Define ARRAY 11`

`#define PRICE$ 100`

ANY QUESTIONS?

THANK YOU